

equivR: Reproducible Cross-Runtime Equivalence for R Analytics

Yonglin Zhu 

Independent open-source project
zhuygl@gmail.com

Xing Cai 

Independent open-source project
caixingt@gmail.com

Executive Summary

R is becoming an important open-source option for clinical-trial analysis and regulatory submissions, but long-term reviewability remains a practical challenge as packages, runtimes, and dependencies change. Environment preservation helps, but some analytical workflows eventually need to be refactored or migrated rather than frozen in place.

equivR addresses that next step: modernize trusted analytical code while independently verifying that its analytical behavior is preserved.

Generation proposes; validation decides.

Phase 1 will first demonstrate SAS -> R modernization using pregenerated trusted SAS reference artifacts, then apply the same workflow to legacy R -> modern R. Over four months, the project will deliver an open-source **equivR** core under MPL-2.0 with deterministic equivalence validation, structured diagnostics, a model-independent LLM repair loop, R-friendly local/CI workflows, and reproducibility evidence.

Signatories

Project team

Yonglin Zhu — co-PI / co-lead. Senior Staff Scientist at SAS Institute. Led development of a completed SAS-to-Python modernization framework using AI-assisted transformation, hybrid SAS/Python execution, and numerical-equivalence validation. Leads architecture, equivalence-contract design, and validation workflow implementation.

Xing Cai — co-PI / co-lead. Founding member of the R Working Group at PPD, with experience in SAS, R, Python, statistical modeling, and data-science workflows. Co-leads R-community use-case design, benchmark development, testing, documentation, and applied evaluation.

equivR is independent open-source work. It is not sponsored, funded, or endorsed by either co-PI's employer and does not depend on employer-owned code, data, or infrastructure.

Contributors

None at submission time. Community contributions will be invited through the public repository and benchmark process.

Consulted

The design is informed by public R Consortium Submissions Working Group materials ([R Consortium 2026a](#), [2026b](#), [2026c](#)); citation does not imply endorsement.

The Problem

Recent R Consortium discussion on making R submissions reviewable for FDA highlights a practical challenge: reviewers may receive R code but still struggle to recreate the sponsor’s environment, install dependencies, or reproduce submitted results ([R Consortium 2026a](#)). As R becomes a more important open-source option for clinical-trial analysis, reviewability and reproducibility become as important as whether the code runs at all.

The R Submissions Working Group is already addressing this problem through approaches such as containers and WebAssembly, while also planning greater use of AI and automation in future submission pipelines ([R Consortium 2026b](#)).

Preserving an old environment, however, does not solve every long-term problem. R versions and packages change, legacy code becomes difficult to maintain, and organizations eventually need to refactor or migrate analytical workflows. The question then becomes:

Can we change the implementation while preserving the analytical behavior that made the original workflow trustworthy?

Today, teams usually answer this with project-specific comparison scripts and manual review. There is no common infrastructure for defining what must remain equivalent and checking those requirements across legacy R, modern R, SAS, Python, or AI-assisted rewrites.

Large language models make modernization easier, but they cannot be trusted to validate their own output. **equivR** targets this missing layer: LLM-assisted modernization combined with independent equivalence validation, so analytical workflows can evolve while remaining reproducible, reviewable, and trustworthy.

The proposal

Overview

equivR will provide a practical way to modernize analytical code without giving up the trusted behavior of the original workflow.

The first target is SAS -> R. A trusted SAS workflow provides reference artifacts; an LLM produces or repairs an R implementation; and **equivR** independently validates the R outputs against the SAS reference. Failed checks return structured diagnostics for another repair attempt. A candidate is accepted only when all declared checks pass.

The same framework will then be demonstrated on legacy R -> modern R. This shows that the approach supports both cross-runtime migration and long-term maintenance within R.

Over four months we will deliver the equivalence contract, deterministic validation core, both benchmark workflows, a model-independent repair loop, an R-friendly local/CI interface, documentation, and an initial release.

Detail

The project separates code generation from acceptance. An LLM may propose a refactor, translation, or repair; only deterministic `equivR` validation decides whether the result preserves the trusted analytical behavior.

Figure 1 summarizes the workflow.

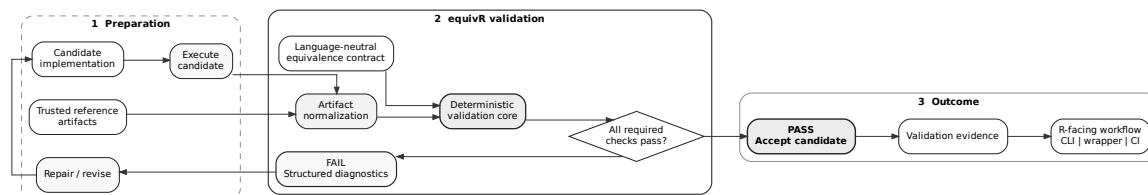


Figure 1: `equivR` modernization and validation workflow. An LLM (or developer) proposes a candidate implementation, which is executed and normalized alongside trusted reference artifacts. The deterministic core applies the equivalence contract and accepts the candidate only when all required checks pass; otherwise it returns structured diagnostics for repair and re-validation.

Minimum Viable Product

The Phase 1 MVP will let a user:

1. provide trusted reference artifacts and an R candidate;
2. declare an equivalence contract covering keys, schema, missingness, categorical values, numerical tolerances, and selected analytical results;
3. run deterministic validation locally or in CI and receive PASS/FAIL status with structured diagnostics; and
4. return failed diagnostics to an LLM for bounded repair and re-validation.

Two public examples will be delivered:

- SAS -> R, using pregenerated SAS reference artifacts so SAS is not required in CI; and
- legacy R -> modern R, using the same validation framework.

Architecture

Phase 1 has four components:

1. **LLM modernization interface** — model-independent generation and repair.
2. **Equivalence contract** — machine-readable definition of what behavior must be preserved.
3. **Runtime/artifact layer** — execute the R candidate and normalize candidate/reference outputs.
4. **Deterministic validator and evidence** — apply the contract, block acceptance on any required failure, and emit diagnostics plus reproducibility evidence.

An R-friendly command or thin wrapper will expose the workflow without requiring users to interact directly with orchestration internals.

Assumptions

- Trusted reference artifacts are available.
- Important behavior can be represented through datasets and selected analytical results.
- Acceptable differences can be stated explicitly, including tolerances where exact equality is inappropriate.
- General-purpose LLMs can produce useful candidate transformations when given source code and validation diagnostics.

If an LLM cannot complete a transformation, **equivR** can still validate a human-written or externally generated candidate. Unsupported behavior will be documented rather than treated as verified.

External dependencies

The project will use open-source R/Python libraries, standard CI tooling, and existing general-purpose LLMs. The automated loop requires an LLM but not a particular vendor or model family.

SAS is needed only to generate the initial trusted reference artifacts; public CI will use pregenerated artifacts and will not require licensed SAS software. Proprietary model API costs used during development or evaluation will be self-funded or provided in kind.

Project plan

The project will run for four months. We will build the deterministic validation core first, demonstrate it on SAS $\rightarrow$ R, then apply the same workflow to legacy R $\rightarrow$ modern R, and finish with the repair loop, documentation, and release.

Start-up phase

Month 1. Finalize governance and MPL-2.0 licensing, define the first equivalence-contract format, implement the initial deterministic validator, and prepare the SAS $\rightarrow$ R benchmark with pregenerated trusted reference artifacts.

By the end of Month 1, **equivR** should accept a reference artifact, an R candidate result, and an equivalence contract, then return deterministic PASS/FAIL status with structured diagnostics.

Technical delivery

Milestone	Timing	Deliverable	Amount
M1	Month 1-2	Equivalence contract, deterministic dataset/result validation, diagnostics, tests	\$1,250
M2	Month 1-2	SAS -> R modernization demonstration using pregenerated SAS reference artifacts	\$1,000
M3	Month 2-3	Legacy R -> modern R demonstration, R-friendly invocation, CI workflow, benchmark format	\$1,000
M4	Month 3-4	Model-independent bounded LLM repair loop and reproducibility evidence	\$750
M5	Month 4	Documentation, examples, release hardening, and initial tagged release	\$1,000

Failure recovery. If an LLM cannot produce a correct candidate, we will narrow or manually repair the example while preserving the validation experiment. Unsupported SAS/R behavior will be documented rather than treated as verified. Model-provider failure will not affect deterministic CI, and the repair loop will stop after a bounded number of attempts.

Other aspects

Development will be public under MPL-2.0, with feedback and benchmark contributions through the repository. Phase 1 remains intentionally narrow: two small benchmark cases, artifact-based validation, and no requirement for live SAS in CI.

Budget & funding plan

Total requested: \$5,000. The milestone amounts are shown above.

No grant funds are requested for travel, workshops, hardware, publication fees, cloud services, AI credits, indirect costs, or licensed SAS software.

Success

Definition of done

Phase 1 is complete when both SAS -> R and legacy R -> modern R workflows run from a clean checkout, use explicit equivalence contracts, return deterministic diagnostics, and produce reproducible validation evidence.

A live SAS runtime will not be required in CI. The automated repair loop will be model/provider independent, and the LLM will never determine acceptance.

Measuring success

Success means:

- both benchmark cases reproduce from a clean checkout;
- every declared required mismatch fails with actionable diagnostics;

- the SAS -> R example validates against pregenerated trusted artifacts;
- an R-friendly invocation path and CI example are documented;
- reproducibility metadata and machine-readable evidence are produced; and
- an initial public release is published.

The goal is not to show that an LLM can translate every analytical program. The goal is to show that modernization can be made verifiable: generation may vary, but acceptance remains deterministic.

Future work

Future work may extend the framework to additional SAS -> R and Python -> R cases, richer analytical result types, live runtime adapters, repository-scale modernization, and more automated multi-step repair. The benchmark format can also grow through community contributions and future R Consortium submission-oriented use cases.

- R Consortium. 2026a. “Making R Submissions Reviewable for FDA.” 2026. <https://r-consortium.org/posts/making-r-submissions-reviewable-for-fda/>.
- . 2026b. “R Submissions Working Group: 2026 Plans.” 2026. <https://r-consortium.org/posts/submissions-wg-2026/>.
- . 2026c. “Submissions Pilot 5 Dataset-JSON Repository.” 2026. <https://github.com/RConsortium/submissions-pilot5-datasetjson>.